

BP-LCGCD: A Gaussian-Elimination-Free and High-Performance Decoder for Surface Codes

Qianfan Wang¹, Member, IEEE, Jifan Liang², Lvzhou Li¹,
Linqi Song¹, Senior Member, IEEE, and Xiao Ma¹, Member, IEEE

Abstract—Belief propagation (BP) combined with ordered statistics decoding (OSD) can achieve near-optimal logical error rates for surface codes. However, OSD requires high-latency and unstable-complexity Gaussian elimination (GE), limiting its practicality. In this letter, we propose BP-LCGCD, a GE-free and high-performance decoder that replaces the GE-based OSD with the GE-free LC-GCD. Moreover, in contrast to the original BP-OSD, which adopts a single fixed normalization factor α , we further propose a list-parameterized variant, BP-LCGCD+ α , which performs multiple BP decodings with different α to generate diverse posterior LLRs. We present complexity analysis, demonstrating that at low physical error rates, the average decoding complexity of the proposed algorithm approaches that of standard BP. Simulation results demonstrate that BP-LCGCD achieves logical error rates close to BP-OSD, while BP-LCGCD+ α nearly matches the performance of the BP-OSD. They also show that both proposed decoders significantly outperform standard BP and minimum-weight perfect matching (MWPM) in terms of logical error rate and threshold.

Index Terms—Locally constrained guessing codeword decoding (LC-GCD), logical error rate, OSD, surface codes.

I. INTRODUCTION

QUANTUM computing can transcend the information-processing limits of classical systems, offering exponential speedups for important tasks such as integer factorization and quantum simulation [1], [2], [3]. However, physical qubits, the core carriers of quantum computing, are highly susceptible to decoherence and operational errors. Quantum error-correction codes (QECCs) are thus essential for protecting physical qubits and realizing fault-tolerant quantum computation [4], [5].

Surface codes, a planar variant of Kitaev's toric code, have emerged as one of the leading classes of QECCs due to

their high error thresholds and near-neighbor layouts [6], [7]. Recent experiments further demonstrate that the logical error rate can be reduced by increasing the code distance [8], [9]. These results indicate that increasing the code distance systematically suppresses logical errors, reinforcing the promise of surface codes as a platform for fault-tolerant quantum computation.

For the decoding of the surface codes, the minimum-weight perfect matching (MWPM) is a widely used algorithm that solves a graph-matching problem to identify the error patterns, and achieves good performance [10], [11]. Other graph-matching-based decoders have also been explored, such as spanning tree matching decoders [12] or almost-linear-time (or linear-time) decoders [13], [14], which reduce complexity compared to MWPM at the cost of some performance loss. Alternatively, BP decoding, originally developed for classical LDPC codes, has also been adapted to surface codes [15], [16], [17]. While BP offers low complexity and low latency, it suffers from significant performance loss compared to MWPM.

To mitigate this loss, OSD has been applied as a post-processor to BP, resulting in the original BP-OSD algorithm (hereafter referred to as BP-OSD) [18]. In this framework, BP is performed at the pre-processing stage to provide soft information, enabling OSD to perform re-encoding based on the most reliable basis (MRB). Empirically, BP-OSD significantly outperforms both MWPM (as well as almost-linear-time decoders) and BP decoder under depolarizing channels, especially at low physical error rate [18], [19], [20], [21]. However, OSD requires Gaussian elimination (GE) to extract the MRB, which introduces unpredictable latency and incurs $\mathcal{O}(n^3)$ time and $\mathcal{O}(n^2)$ memory complexity [22], rendering it unsuitable for the low-latency requirements of quantum error correction systems.

A key question thus arises: can we eliminate GE yet retaining near-optimal decoding performance? The answer is positive and can be achieved by the LC-GCD algorithm [23], [24], which circumvents GE by a fixed basis and preserves decoding performance by the locally-constrained searches. In this work, we take the LC-GCD as the post-processor to BP, resulting in a GE-free decoder, called BP-LCGCD. Moreover, the normalization factor in NMS significantly affects the posterior LLRs and decoding performance, but original BP-OSD uses only a single fixed α , which constrains its search space. To address this, we further propose a parameter-listized enhancement, called BP-LCGCD+ α , to improve the performance. Our main contributions are: i) **BP-LCGCD**: We replace the GE-based OSD stage in BP-OSD with LC-GCD to obtain BP-LCGCD, eliminating GE steps while retaining

Received 23 November 2025; revised 11 December 2025; accepted 16 December 2025. Date of publication 22 December 2025; date of current version 9 January 2026. This work was supported by the National Key Research and Development Program of China (2024YFB4504004), the NSFC (62301617, 62471506 and 62371411), Guangdong Basic and Applied Basic Research Foundation (2023A1515011056 and 2025A1515011650), Young Talent Support Project of Guangzhou Association for Science and Technology (QT-2025-048), the Guangdong Provincial Quantum Science Strategic Initiative (GDZX2303007). The associate editor coordinating the review of this letter and approving it for publication was M. Battaglini. (Corresponding authors: Linqi Song; Xiao Ma.)

Qianfan Wang is with the Department of Computer Science, City University of Hong Kong, Hong Kong, SAR (e-mail: qwang742@cityu.edu.hk).

Jifan Liang, Lvzhou Li, and Xiao Ma are with the School of Computer Science and Engineering, Sun Yat-sen University, Guangzhou 510006, China (e-mail: liangjif56@mail2.sysu.edu.cn; lilvzh@mail.sysu.edu.cn; maxiao@mail.sysu.edu.cn).

Linqi Song is with the Department of Computer Science, City University of Hong Kong, Hong Kong, SAR, and also with the City University of Hong Kong Shenzhen Research Institute, Shenzhen, Guangdong 518057, China (e-mail: linqi.song@cityu.edu.hk).

Digital Object Identifier 10.1109/LCOMM.2025.3646724

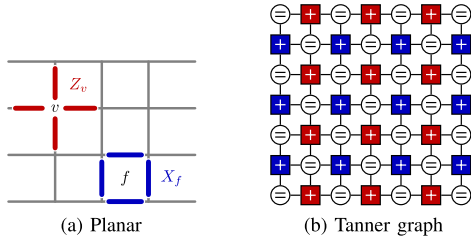


Fig. 1. The planar and Tanner graph of surface codes. In planar, edges represent physical qubits, faces represent X -type stabilizers, and vertices represent Z -type stabilizers. In Tanner graph, $\ominus$ indicate physical qubits, blue $\oplus$ indicate X -type stabilizers, and red $\oplus$ indicate Z -type stabilizers.

good logical error rates; ii) **Multi- α Improvement:** We extend BP-LCGCD to a generalized list decoder, BP-LCGCD+ α , by performing multiple BP decodings with different normalization factors α and selectively applying LC-GCD to generate candidates; and iii) **Complexity:** We provide theoretical analysis for decoding complexity, showing that the proposed algorithm maintains a complexity comparable to BP at low error rates.

Numerical results show that BP-LCGCD achieves logical error rates close to BP-OSD, while BP-LCGCD+ α nearly matches the performance of the original BP-OSD. They also show that the proposed decoders significantly outperform BP and MWPM in terms of logical error rate and threshold.

II. PRELIMINARIES

A. Stabilizer Codes and Surface Codes

Let $\mathcal{G}_n = \{\pm 1, \pm i\} \times \{I, X, Y, Z\}^{\otimes n}$ be the n -qubit Pauli group, where I, X, Y , and Z are the basic Pauli operators. Let $\mathcal{C}[[n, k, d]]$ denote a stabilizer code, which encodes k logical qubits into n physical qubits with distance d , correcting up to $\lfloor (d-1)/2 \rfloor$ errors. Stabilizer code $\mathcal{C}$ is specified by an abelian subgroup $\mathcal{S} = \langle S_1, \dots, S_m \rangle \subset \mathcal{G}_n$, $m \geq n - k$, called the *stabilizer group* [4]. Define the stabilizer matrix $\mathbf{S}$ of size $m \times n$, where each row corresponds to a stabilizer generator S_i . The code space is the 2^k -dimensional subspace

$$\mathcal{C} = \{|\psi\rangle : S_i|\psi\rangle = |\psi\rangle, \forall S_i \in \mathcal{S}\}.$$

For decoding, each S_i is measured to obtain syndrome bits $s_i \in \mathbb{F}_2$, after which an error pattern is inferred from $\mathbf{s} \in \mathbb{F}_2^m$.

A planar surface code $[[2d^2 - 2d + 1, 1, d]]$ is realized on a $d-1$ order square lattice (see Fig. 1(a)). Physical qubits reside on edges, and there are two types of stabilizer: i) X -type on each face f : $X_f = \prod_{e \in \partial f} X_e$, and ii) Z -type on each vertex v : $Z_v = \prod_{e: v \in \partial e} Z_e$, where ∂f is the set of edges around face f , and ∂e is the two endpoints of edge e . All X - and Z -type stabilizers commute, i.e., $[X_f, X_{f'}] = [Z_v, Z_{v'}] = [X_f, Z_v] = 0$. Hence, all generators can be measured simultaneously [25]. As a special class of quantum LDPC codes, the surface code can also be represented by a Tanner graph $(\mathcal{I}, \mathcal{J}, \mathcal{E})$: variable nodes $\mathcal{I}$ ($|\mathcal{I}| = n$) for qubits, check nodes $\mathcal{J}$ ($|\mathcal{J}| = m$) for stabilizer generators, and an edge $(i, j) \in \mathcal{E}$ if stabilizer S_j acts nontrivially on qubit i (Fig. 1(b)).

B. Quantum BP-OSD Algorithm

In quantum BP-OSD, BP is performed at pre-processing stage to produce posterior LLRs, and OSD is performed at post-processing stage to search candidate error patterns. The BP stage performs joint iterative decoding of X and Z errors on the Tanner graph, capturing error correlations with modest complexity overhead [26]. In contrast, the post-processing LC-GCD stage decodes X and Z errors separately, in line with the original LC-GCD approach. We now detail each component.

1) **Quantum BP:** We adopt the NMS variant of quantum BP with normalization factor α , which is executed on the Tanner graph $(\mathcal{I}, \mathcal{J}, \mathcal{E})$ to compute posterior probabilities of Pauli errors. Unlike classical BP, each check node j holds a syndrome $s_j \in \{0, 1\}$. Hence, define $(\hat{\mathbf{E}}, \boldsymbol{\mu}) = \text{NMS}(\alpha, \boldsymbol{\Lambda}, \mathbf{s})$, where $\boldsymbol{\Lambda}$ denotes the prior LLR vector, $\hat{\mathbf{E}}$ is the hard-decision error pattern, and $\boldsymbol{\mu}$ is the posterior LLR vector. The prior LLRs $\boldsymbol{\Lambda}$ for qubit i are

$$\Lambda_i = (\Lambda_i^X, \Lambda_i^Y, \Lambda_i^Z), \quad \Lambda_i^P = \ln(p_i^I / p_i^P),$$

where p_i^P is the prior probability of Pauli $P \in \{X, Y, Z\}$ on qubit i , and $p_i^I = 1 - \sum_{P \neq I} p_i^P$. We first initialize

$$\mu_{i \rightarrow j}^P(0) = \Lambda_i^P, \quad \forall (i, j) \in \mathcal{E}, P \in \{X, Y, Z\}.$$

The t -th iteration proceeds as follows.

1) Update Check-to-Variable Message $\mu_{j \rightarrow i}$:

$$\begin{aligned} \mu_{j \rightarrow i}(t) &= (-1)^{s_j} \alpha \prod_{i' \in \mathcal{I}(j) \setminus i} \text{sgn}(\lambda_{i' \rightarrow j}(t-1)) \min_{i' \in \mathcal{I}(j) \setminus i} |\lambda_{i' \rightarrow j}(t-1)| \end{aligned}$$

for $j \in \mathcal{J}$, $i \in \mathcal{I}(j)$, where $\mathcal{I}(j)$ is the set of variable nodes adjacent to check j . Then, $\lambda_{i \rightarrow j}(t)$ is computed from the outgoing variable-to-check messages as

$$\lambda_{i \rightarrow j} = \log \frac{1 + \exp(-\mu_{i \rightarrow j}^P)}{\sum_{Q \in \{X, Y, Z\} \setminus P} \exp(-\mu_{i \rightarrow j}^Q)},$$

where $P = \mathbf{S}_{j,i}$ is the Pauli type given by the stabilizer matrix entry at (j, i) , and $j \in \mathcal{J}(i)$.

2) **Hard Decision:** Compute the total incoming message to variable i : $\mu_i^P(t) = \Lambda_i^P + \sum_{j \in \mathcal{J}(i), \langle P, \mathbf{S}_{j,i} \rangle = 1} \mu_{j \rightarrow i}(t)$, $P \in \{X, Y, Z\}$, $i \in \mathcal{I}$. Then make the hard decision for qubit i as follows: set $\hat{E}_i = I$ if $\min\{\mu_i^X, \mu_i^Y, \mu_i^Z\} \geq 0$; otherwise set $\hat{E}_i = \arg \min_{P \in \{X, Y, Z\}} \mu_i^P$. If the hard decision satisfies $\langle \hat{\mathbf{E}}, \mathbf{S}_j \rangle = s_j$, $\forall j \in \mathcal{J}$, or if the maximum iteration is reached, decoding terminates.

3) **Update Variable-to-Check Message $\mu_{i \rightarrow j}$:** $\mu_{i \rightarrow j}^P(t) = \mu_i^P(t) - \langle P, \mathbf{S}_{j,i} \rangle \mu_{j \rightarrow i}(t)$, for $P = \mathbf{S}_{j,i}$, $i \in \mathcal{I}$, $j \in \mathcal{J}(i)$.

To improve performance, OSD is applied after BP. Classical OSD reorders bits by reliability, applies GE to obtain the MRB, and tests low-weight error patterns on the MRB. For quantum codes, the decoding problem is decoupled for X and Z errors. Let $\mathbf{E}_X, \mathbf{E}_Z \in \{0, 1\}^n$ denote the binary indicator vectors for X - and Z -type errors, respectively. Correspondingly, we partition the parity-check matrix as $\mathbf{H} = [\mathbf{H}_X | \mathbf{H}_Z]$, where $\mathbf{H}_X$ checks Z -type errors and $\mathbf{H}_Z$ checks X -type errors. We define the bilinear form $\odot$ for syndrome computation as $\mathbf{s} \triangleq \mathbf{E} \odot \mathbf{H} = \mathbf{E}_X \mathbf{H}_Z^T + \mathbf{E}_Z \mathbf{H}_X^T$.

(mod 2), where $\mathbf{E} = (\mathbf{E}_X, \mathbf{E}_Z)$. Quantum OSD applies classical OSD to this binary formulation to find the most likely error pattern [18], [19]. Unlike MRB-flip OSD, which orders candidates by Hamming weight over the MRB and enumerates flip subsets, our implementation uses BP soft information to score and rank TEPs on the extended basis of length $k + \delta$, and the top $\ell_{\max}$ TEPs are then re-encoded to yield syndrome-consistent candidates. However, OSD requires GE, which introduces unpredictable latency and $O(n^3)$ complexity. Even worse, GE is inherently serial¹ and difficult to parallelize, resulting in high latency. In the next subsection, we present the GE-free LC-GCD as a more practical post-processing decoder.

C. LC-GCD and Its Quantum Extension

1) *Classical LC-GCD*: Let $\mathbf{H} = [\mathbf{I} \mid \mathbf{P}] \in \mathbb{F}_2^{(n-k) \times n}$ be the systematic parity-check matrix. A test error pattern (TEP), written as $\mathbf{e} = (\mathbf{e}_I, \mathbf{e}_P)$ with $\mathbf{e}_I \in \mathbb{F}_2^{n-k}$ and $\mathbf{e}_P \in \mathbb{F}_2^k$, satisfies $\mathbf{e}_I + \mathbf{e}_P \mathbf{P}^T = \mathbf{s}$. The hard-decision sequence is correspondingly written as $\mathbf{z} = (\mathbf{z}_I, \mathbf{z}_P)$. In contrast to traditional OSD- or GCD-like decoders that search over $\mathbf{e}_P \in \mathbb{F}_2^k$, LC-GCD searches over a space of dimension $k + \delta$, skipping many invalid TEPs. The algorithm proceeds:

- 1) *Preprocessing*: Select an index set $\mathcal{A} \subseteq \{1, \dots, n-k\}$ of size δ corresponding to the most reliable positions in $\mathbf{e}_I$, and let $\mathcal{A}^c$ be its complement. Denote by $\mathbf{P}_{\mathcal{A}}$ and $\mathbf{P}_{\mathcal{A}^c}$ the submatrices of $\mathbf{P}$ with rows indexed by $\mathcal{A}$ and $\mathcal{A}^c$, respectively. For any valid TEP $\mathbf{e}$, we have

$$\mathbf{e}_I[\mathcal{A}] + \mathbf{e}_P \mathbf{P}_{\mathcal{A}}^T = \mathbf{z}_I[\mathcal{A}] + \mathbf{z}_P \mathbf{P}_{\mathcal{A}}^T, \quad (1)$$

$$\mathbf{e}_I[\mathcal{A}^c] + \mathbf{e}_P \mathbf{P}_{\mathcal{A}^c}^T = \mathbf{z}_I[\mathcal{A}^c] + \mathbf{z}_P \mathbf{P}_{\mathcal{A}^c}^T, \quad (2)$$

- 2) *List Search*: Build a trellis from $[\mathbf{I}_{\mathcal{A}} \mid \mathbf{P}_{\mathcal{A}}]$ using (1) and perform the SLVA [29] to search $(\mathbf{e}_I^{(\ell)}[\mathcal{A}], \mathbf{e}_P^{(\ell)})$ in order of increasing partial soft weight. For each candidate, recover $\mathbf{e}_I^{(\ell)}[\mathcal{A}^c]$ via (2), resulting in the ℓ -th TEP $\mathbf{e}^{(\ell)}$.

2) *Quantum LC-GCD*: Quantum LC-GCD applies the same LC search to TEPs, but operates on the binary check matrix $\mathbf{H} = [\mathbf{H}_X \mid \mathbf{H}_Z]$ along with the measured syndrome. Each candidate error pattern $\mathbf{Q}^{(\ell)}$ is reconstructed as

$$\mathbf{Q}^{(\ell)} = \mathbf{Q}_{\text{MRB}}^{(\ell)} \mathbf{H}_{\text{enc}}^T + \mathbf{Q}^{(0)}, \quad 1 \leq \ell \leq \ell_{\max},$$

where $\mathbf{H}_{\text{enc}}$ is the encoding matrix that maps an augmented error-pattern vector of length $k + \delta$ (obtained from the local constraints) to a length- n candidate error pattern guaranteed to match the measured syndrome [30]. The final output is the candidate of minimum total weight (equivalently, highest likelihood) under the depolarizing channel.

Remark. Unlike GE-based OSD in BP-LCOSD [30], our LC-GCD enumerates locally constrained flips from a fixed generator basis and therefore avoids GE. To enhance performance, BP-LCGCD+ α aggregates candidates produced by multiple BP front-ends using distinct normalization factors α .

III. BP-LCGCD ALGORITHM

In the original BP-OSD framework, BP is performed on the sparse Tanner graph to produce an estimated error pattern

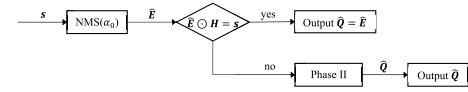
¹For certain specially constructed codes, such as staircase matrix codes, GE can be parallelized; see [27], [28].

Algorithm 1 List-Parameterized BP-LCGCD Algorithm

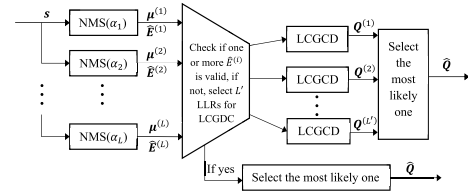
Require: LLR vector Λ ; syndrome $\mathbf{s}$; parity-check matrix $\mathbf{H}$; initial factor α_0 ; list factors $\{\alpha_t\}_{t=1}^L$; L' ; $\ell_{\max}$

Ensure: $\hat{\mathbf{Q}}$

- 1: **Phase I (single α_0 BP):**
- 2: $(\hat{\mathbf{E}}^{(0)}, \boldsymbol{\mu}^{(0)}) \leftarrow \text{NMS}(\alpha_0, \Lambda, \mathbf{s})$
- 3: **if** $\hat{\mathbf{E}}^{(0)} \odot \mathbf{H} = \mathbf{s}$ **then**
- 4: **return** $\hat{\mathbf{E}}^{(0)}$
- 5: **Phase II (list- α BP + LC-GCD):**
- 6: Initialize $\mathcal{S} \leftarrow \emptyset$, $\mathcal{Q} \leftarrow \emptyset$
- 7: **for** $t = 1$ **to** L **do**
- 8: $(\hat{\mathbf{E}}^{(t)}, \boldsymbol{\mu}^{(t)}) \leftarrow \text{NMS}(\alpha_t, \Lambda, \mathbf{s})$
- 9: **if** $\hat{\mathbf{E}}^{(t)} \odot \mathbf{H} = \mathbf{s}$ **then**
- 10: $\mathcal{S} \leftarrow \mathcal{S} \cup \{\hat{\mathbf{E}}^{(t)}\}$
- 11: **if** $\mathcal{S} \neq \emptyset$ **then**
- 12: **return** $\arg \min_{\mathbf{E} \in \mathcal{S}} \text{wt}(\mathbf{E})$
- 13: **Select top- L' outputs:** choose $\mathcal{T} \subseteq \{1, \dots, L\}$ with $|\mathcal{T}| = L'$ to maximize $\sum_{t \in \mathcal{T}} \sum_{i \in I_P \cup \mathcal{A}} |\mu_i^{(t)}|$
- 14: **for** $t \in \mathcal{T}$ **do**
- 15: $\mathbf{Q}^{(t)} \leftarrow \text{LC-GCD}(\boldsymbol{\mu}^{(t)}, \ell_{\max})$
- 16: $\mathcal{Q} \leftarrow \mathcal{Q} \cup \mathbf{Q}^{(t)}$
- 17: **Final selection:** $\hat{\mathbf{Q}} \leftarrow \arg \min_{\mathbf{Q} \in \mathcal{Q}} \text{wt}(\mathbf{Q})$
- 18: **return** $\hat{\mathbf{Q}}$



(a) Overall diagram of the proposed decoding algorithm.



(b) Diagram of phase II.

Fig. 2. Diagram of the proposed BP-LCGCD algorithm.

and posterior reliabilities. If BP fails, these reliabilities are fed into an OSD module for post-processing. In our BP-LCGCD algorithm, we (i) replace the GE-based OSD with GE-free LC-GCD, and (ii) expand the BP search space by list-parameterizing the normalization factor α . In this section, we present the BP-LCGCD algorithm and its complexity analysis.

A. Algorithm Overview

Recall that Λ is the prior LLR vector and $\mathbf{s} \in \{0, 1\}^m$ is the measured syndrome. We select a primary normalization factor α_0 and a list $\{\alpha_t\}_{t=1}^L$. The proposed BP-LCGCD operates in two phases, as shown in Fig. 2(a). In Phase I (single- α_0 BP), we execute NMS decoding with factor α_0 . If the resulting hard decision satisfies $\hat{\mathbf{E}}^{(0)} \odot \mathbf{H} = \mathbf{s}$, then output $\hat{\mathbf{Q}} = \hat{\mathbf{E}}^{(0)}$ and stop the decoding. Otherwise, we enter Phase II (list- α BP + LC-GCD), as shown in Fig. 2(b), which proceeds as follows:

- a) Perform NMS for each α_t , $t = 1, \dots, L$, and collect all $\hat{\mathbf{E}}^{(t)}$ satisfying $\hat{\mathbf{E}}^{(t)} \odot \mathbf{H} = \mathbf{s}$.

- b) If any BP run succeeds, output the minimum-Hamming-weight estimate $\hat{Q}$.
- c) Otherwise, compute $\lambda_t = \sum_{i \in I_P \cup A} |\mu_i^{(t)}|$ for $1 \leq t \leq L$ and select the top L' sequences with the largest λ_t .
- d) For each of these L' sequences, invoke LC-GCD with list size $\ell_{\max}$ to generate candidate error patterns.
- e) From all candidates produced by LC-GCD, pick the minimum-weight pattern as the final estimate $\hat{Q}$.

The BP-LCGCD algorithm is summarized in Algorithm 1 (see Fig. 2 for reference).

B. Complexity Analysis

The overall complexity of the list-parameterized BP-LCGCD+ α algorithm consists of two main components: BP and LC-GCD. The complexity analysis is as follows.

1) *BP Complexity*: Performing NMS on the Tanner graph requires on average $\mathcal{O}(nT_{\text{avg}})$ operations, where T_{avg} is the average number of iterations. In our two-phase framework, let $T_{1,\text{avg}}$ and $T_{2,\text{avg}}$ denote the average iterations in Phase I and Phase II, respectively. Then

Phase I BP cost: $\mathcal{O}(nT_{1,\text{avg}})$, Phase II BP cost: $\mathcal{O}(LnT_{2,\text{avg}})$.

2) *LC-GCD Complexity*: Let δ be the degree of the local constraints, and ℓ_{avg} be the average list size of LC-GCD. An approximation of the average cost of one LC-GCD is $\mathcal{O}(\ell_{\text{avg}}kn + 2^\delta(k + \delta))$.

3) *Total Average Complexity*: Let η_1 be the probability that Phase I BP fails, and let η_2 be the probability that all L BP runs in Phase II fail (requiring LC-GCD). By selecting the top L' sequences for LC-GCD, the average complexity is

$$\begin{aligned} &\mathcal{O}(nT_{1,\text{avg}}) + \eta_1[\mathcal{O}(LnT_{2,\text{avg}}) \\ &+ \eta_2\mathcal{O}(L'(\ell_{\text{avg}}kn + 2^\delta(k + \delta)))]. \end{aligned} \quad (3)$$

In the low physical error-rate regime, Phase I succeeds with high probability, making $\eta_1 = \Pr\{\text{Phase II is invoked}\} \ll 1$. Consequently, the average complexity is dominated by the single BP run in Phase I, approaching $\mathcal{O}(nT_{1,\text{avg}})$, while the contribution from Phase II becomes negligible. For example, at $p \approx 10^{-3}$ on the tested codes we observe $\eta_1 \approx 10^{-4}$, i.e., Phase II (LC-GCD) is invoked roughly once in 10^4 trials. Note that the same conclusion applies to BP-OSD, whose post-processing is invoked only upon a Phase I failure. In the worst case, we have $\eta_1 = \eta_2 = 1$ and ℓ , T_1 , T_2 taking their maximum values in (3), so that the total decoding complexity is governed by the corresponding worst-case bounds on the BP and LC-GCD stages. Also note that, because the worst-case complexity of the post-processing scales exponentially with the code dimension k , the proposed approach may become impractical for higher-rate QLDPC codes.

IV. NUMERICAL RESULTS

In this section, we evaluate the proposed BP-LCGCD algorithm over the quantum depolarizing channel using Monte Carlo simulations. Unless otherwise specified, we set the LC degree to $\delta = 8$, use $L = 16$ normalization factors α , and choose $L' = 8$ for LC-GCD. The main decoding parameters adopted in our experiments are summarized in Table I.

TABLE I
DECODING PARAMETERS

Parameter	Symbol	Value
Max BP iterations	$T_{\max}$	32
Phase I normalization	α_0	5/8
Phase II normalization set	$\{\alpha_t\}$	$\{1/8, 2/8, \dots, 2\}$
LC-GCD list size	$\ell_{\max}$	2^{10}
Degree of local constraints	δ	8
Depolarizing rate	p	10^{-3} – $10^{-0.5}$

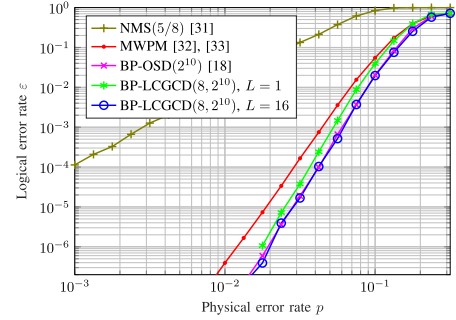


Fig. 3. Decoding performance for surface code $[[145, 1, 9]]$, where we compare 1) NMS; 2) MWPM; 3) BP-OSD; 4) BP-LCGCD.

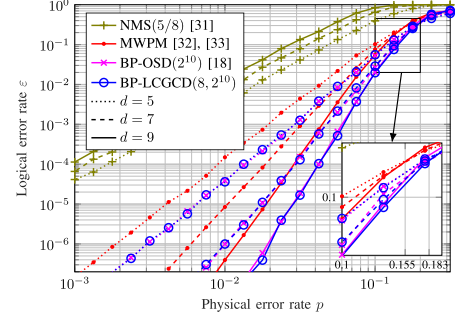


Fig. 4. Logical error rate for surface codes of distance $d = 5, 7, 9$, where we compare 1) NMS; 2) MWPM; 3) BP-OSD; 4) BP-LCGCD+ α .

Example 1 (Surface code $[[145, 1, 9]]$): We consider the surface code $[[145, 1, 9]]$ as in [6], under the depolarizing channel. We simulate four decoders: 1) NMS with $\alpha = 5/8$ [31]; 2) MWPM [32], [33]; 3) BP-OSD [18]; 4) BP-LCGCD with $\delta = 8$, and $L = 1$ or 16. As shown in Fig. 3, we observe that the BP-LCGCD algorithm significantly outperforms BP and MWPM. As L increases beyond 1, the proposed BP-LCGCD+ α exhibits nearly identical performance to that of BP-OSD, demonstrating the gain from list-parameterization.

Example 2 (Surface codes with different distances): We consider the surface codes [6] with distances $d = 5, 7, 9$, under the same depolarizing channel. Four decoders are compared: 1) NMS with $\alpha = 5/8$ [31]; 2) MWPM [32], [33]; 3) BP-OSD [18]; 4) BP-LCGCD+ α with $\delta = 8$ and $L = 16$. The logical error rate is shown in Fig. 4, from which we observe that the proposed BP-LCGCD+ α algorithm consistently outperforms both NMS and MWPM across all considered code sizes, and achieves nearly identical performance to the performance of BP-OSD. In particular, the proposed decoder exhibits a significantly higher threshold than NMS and MWPM, closely matching the BP-OSD threshold of about 18%.

Example 3 (Hexagonal color codes): Consider the 6.6.6 color codes [34] with $d = 5, 7, 9$ under the

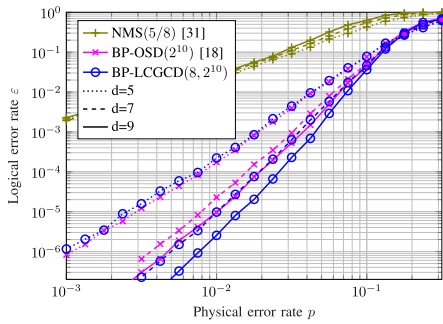


Fig. 5. Logical error rate for 6.6.6 (hexagonal) color codes of $d = 5, 7, 9$, where we compare 1) NMS; 2) BP-OSD; 3) BP-LCGCD+ α .

depolarizing channel. We evaluate three decoders: (i) NMS with $\alpha = 5/8$ [31]; (ii) BP-OSD [18]; and (iii) the proposed BP-LCGCD+ α with $\delta = 8$ and $L = 16$. As shown in Fig. 5, for all tested codes, the proposed algorithm outperforms NMS and achieves performance comparable or slightly superior to BP-OSD.

V. CONCLUSION

We have proposed BP-LCGCD, a GE-free list decoder for surface codes, which replaces the OSD module in BP-OSD with LC-GCD, thereby eliminating online GE. To further enhance decoding performance, we introduced BP-LCGCD+ α , which performs multiple BP decodings with distinct normalization factors and applies LC-GCD to generate candidates. We also presented a complexity analysis, showing that the proposed decoding algorithm can achieve complexity comparable to standard BP decoding under low physical error rates. Numerical results show that the proposed decoders achieve logical error rates and thresholds nearly identical to those of BP-OSD, while significantly outperforming BP and MWPM. As a promising direction for future work, we plan to evaluate BP-LCGCD under more realistic noise models, including phenomenological and circuit-level models [13], [35], [36], to assess robustness to circuit-induced correlations and leakage.

REFERENCES

- [1] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.*, 1994, pp. 124–134.
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*. Cambridge, U.K.: Cambridge Univ. Press, 2010.
- [3] S. Zhang and L. Li, "A brief introduction to quantum algorithms," *CCF Trans. High Perform. Comput.*, vol. 4, no. 1, pp. 53–62, Mar. 2022.
- [4] D. Gottesman, "Stabilizer codes and quantum error correction," Ph.D. dissertation, Dept. Phys., Math., Astron., California Inst. Technol., Pasadena, CA, USA, 1997.
- [5] S. J. Devitt, W. J. Munro, and K. Nemoto, "Quantum error correction for beginners," *Rep. Prog. Phys.*, vol. 76, no. 7, Jul. 2013, Art. no. 076001.
- [6] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, "Topological quantum memory," *J. Math. Phys.*, vol. 43, no. 9, pp. 4452–4505, Sep. 2002.
- [7] A. Y. Kitaev, "Fault-tolerant quantum computation by anyons," *Ann. Phys.*, vol. 303, no. 1, pp. 2–30, Jan. 2003.
- [8] R. Acharya et al., "Suppressing quantum errors by scaling a surface code logical qubit," *Nature*, vol. 614, no. 7949, pp. 676–681, 2023.
- [9] R. Acharya et al., "Quantum error correction below the surface code threshold," *Nature*, vol. 638, no. 8052, pp. 920–926, 2024.
- [10] V. Kolmogorov, "Blossom V: A new implementation of a minimum cost perfect matching algorithm," *Math. Program. Comput.*, vol. 1, no. 1, pp. 43–67, Jul. 2009.
- [11] A. G. Fowler, "Minimum weight perfect matching of fault-tolerant topological quantum error correction in average $O(1)$ parallel time," 2013, *arXiv:1307.1740*.
- [12] D. Forlivesi, L. Valentini, and M. Chiani, "Spanning tree matching decoder for quantum surface codes," *IEEE Commun. Lett.*, vol. 28, no. 7, pp. 1509–1513, Jul. 2024.
- [13] N. Delfosse and N. H. Nickerson, "Almost-linear time decoding algorithm for topological codes," *Quantum*, vol. 5, p. 595, Dec. 2021.
- [14] D. Forlivesi, L. Valentini, and M. Chiani, "Bubble clustering decoder for quantum topological codes," *IEEE Trans. Commun.*, vol. 73, no. 10, pp. 8470–8483, Oct. 2025.
- [15] J. Old and M. Rispler, "Generalized belief propagation algorithms for decoding of surface codes," *Quantum*, vol. 7, p. 1037, Jun. 2023.
- [16] D. Chytas, M. Pacenti, N. Raveendran, M. F. Flanagan, and B. Vasić, "Enhanced message-passing decoding of degenerate quantum codes utilizing trapping set dynamics," *IEEE Commun. Lett.*, vol. 28, no. 3, pp. 444–448, Mar. 2024.
- [17] T.-H. Huang, T.-A. Hu, and Y.-L. Ueng, "Branch-assisted sign-flipping belief propagation decoding for topological quantum codes based on hypergraph product structure," *IEEE Trans. Quantum Eng.*, vol. 4, pp. 1–15, 2023, Art. no. 2100415.
- [18] P. Panteleev and G. Kalachev, "Degenerate quantum LDPC codes with good finite length performance," *Quantum*, vol. 5, p. 585, Nov. 2021.
- [19] A. deMarti iOlius, P. Fuentes, R. Orús, P. M. Crespo, and J. E. Martinez, "Decoding algorithms for surface codes," *Quantum*, vol. 8, p. 1498, Oct. 2024.
- [20] J. Liang, Q. Wang, L. Li, and X. Ma, "A high-performance list decoding algorithm for surface codes with erroneous syndrome," 2024, *arXiv:2409.06979*.
- [21] J. Liang, Q. Wang, L. Li, L. Song, and X. Ma, "A low-complexity BP-OSD algorithm for quantum LDPC codes," *Eur. Phys. J. Special Topics*, Jun. 2025.
- [22] M. P. C. Fossorier and S. Lin, "Soft-decision decoding of linear block codes based on ordered statistics," *IEEE Trans. Inf. Theory*, vol. 41, no. 5, pp. 1379–1396, Sep. 1995.
- [23] X. Zheng and X. Ma, "A universal list decoding algorithm with application to decoding of polar codes," *IEEE Trans. Inf. Theory*, vol. 71, no. 2, pp. 975–995, Feb. 2025.
- [24] X. Zheng and X. Ma, "Locally constrained guessing codeword decoding of short block codes," in *Proc. IEEE Inf. Theory Workshop (ITW)*, Nov. 2024, pp. 442–447.
- [25] D. A. Lidar and T. A. Brun, *Quantum Error Correction*. Cambridge, U.K.: Cambridge Univ. Press, 2013.
- [26] T. Müller et al., "Improved belief propagation is sufficient for real-time decoding of quantum memory," 2025, *arXiv:2506.01779*.
- [27] Y. Wang, J. Liang, Q. Wang, and X. Ma, "Representative ordered statistics decoding of staircase matrix codes," *IEEE Trans. Commun.*, vol. 73, no. 4, pp. 2148–2158, Apr. 2025.
- [28] Q. Wang, Y. Wang, Y. Wang, J. Liang, and X. Ma, "Random staircase generator matrix codes: Coding theorem, performance analysis, and code design," *IEEE Trans. Inf. Theory*, vol. 71, no. 5, pp. 3497–3509, May 2025.
- [29] N. Seshadri and C.-E.-W. Sundberg, "List Viterbi decoding algorithms with applications," *IEEE Trans. Commun.*, vol. 42, nos. 2–4, pp. 313–323, Feb. 1994.
- [30] J. Liang, Q. Wang, L. Li, and X. Ma, "The BP-LCOSD algorithm for toric codes," in *Proc. IEEE Int. Symp. Inf. Theory Workshops (ISIT-W)*, Jul. 2024, pp. 1–6.
- [31] D. Poulin and Y. Chung, "On the iterative decoding of sparse quantum codes," 2008, *arXiv:0801.1241*.
- [32] O. Higgott, "PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching," *ACM Trans. Quantum Comput.*, vol. 3, no. 3, pp. 1–16, Sep. 2022.
- [33] O. Higgott and C. Gidney, "Sparse blossom: Correcting a million errors per core second with minimum-weight matching," *Quantum*, vol. 9, p. 1600, Jan. 2025.
- [34] S. Koutsoumpas, T. Noszko, H. Sayginel, M. Webster, and J. Roffe, "Colour codes reach surface code performance using vibe decoding," 2025, *arXiv:2508.15743*.
- [35] S. Varsamopoulos, K. Bertels, and C. G. Almudever, "Comparing neural network based decoders for the surface code," *IEEE Trans. Comput.*, vol. 69, no. 2, pp. 300–311, Feb. 2020.
- [36] M. Pacenti, M. F. Flanagan, D. Chytas, and B. Vasić, "Progressive-proximity bit-flipping for decoding surface codes," *IEEE Trans. Commun.*, vol. 73, no. 3, pp. 1419–1430, Mar. 2025.